Documento de Enunciado para Trabalho Prático
Curso: Engenharia Informática

Disciplina: Sistemas Distribuídos

Tema: — Desenvolvimento Web com Java (Servlets + Apache Tomcat)

AgroGest — Sistema de Gestão Agrícola

I. Contexto e Motivação

Um agricultor do **da província de Nampula,** dedica-se à produção agrícola em várias campanhas e necessita de um sistema informático que o ajude a:

- Gerir os **fornecedores** de sementes, fertilizantes, maquinaria e ferramentas;
- Controlar os **cultivos e colheitas** de cada campanha agrícola;
- Vender **produtos em stock** (sementes, fertilizantes, ferramentas) a clientes;
- Emitir **facturas** detalhadas com todos os dados da compra;
- Acompanhar **relatórios** de clientes e vendas.

O sistema **AgroGest** é uma aplicação web que responde a essas necessidades. Deve ser implementado em **Java com Servlets** e publicado no **Apache Tomcat**, usando **listas em memória** (sem base de dados), esta é a restrições do exercício.

II. Objetivos de Aprendizagem

No final do trabalho, o estudante deverá ser capaz de:

1. Estruturar uma aplicação web Java EE/Jakarta EE segundo o padrão **MVC (Model-View-Controller)**.
2. Implementar **Servlets** como controladores, **JSP** (JavaServer Pages) como vista e **modelos POJO (Plain Old Java Objects):** *(Representa a lógica de negócio e os dados da aplicação. São classes Java simples que não dependem de frameworks específicos (possuem apenas atributos privados, construtores, getters e setters). Exemplos: Cliente.java, Produto.java),* como domínio.
3. Gerir **sessões HTTP** (autenticação, carrinho de compras).
4. Aplicar **filtros** para controlo de acesso por perfil.

5. Trabalhar com **listas em memória** (ArrayList, Map, synchronizedList) como camada de persistência temporária.
6. Garantir **transações lógicas** (venda com desconto de stock) através de sincronização.
7. Persistir estado em **ficheiro** (serialização Java).
8. Fazer **deploy** de um WAR no Tomcat, configurar portas e gerir o serviço com systemd.

III. Requisitos Funcionais

Perfis de Utilizador

Perfil	Descrição
Administrador	Conta fixa criada no arranque do sistema. Gere todo o catálogo, campanhas, clientes e vendas.
Cliente	Auto regista-se. Consulta o catálogo, adiciona produtos ao carrinho, efetua compras e vê facturas.

IV. Módulo Administrador

Autenticação
O administrador autentica-se com email e palavra-passe fixos exemplo: (admin@agro.mz / admin234).

Gestão de Fornecedores
Criar, listar e remover fornecedores. Cada fornecedor tem: nome, tipo (SEMENTES, MAQUINARIA, FERTILIZANTES), contacto, província.

Gestão de Produtos
Criar, listar, atualizar (repor stock) e remover produtos. Cada produto tem: nome, categoria (SEMENTE, FERTILIZANTE, MAQUINARIA, FERRAMENTA), fornecedor, preço, quantidade, unidade, descrição, foto (URL), validade.

Gestão de Campanhas
Criar campanhas agrícolas com nome e datas de início/fim. Fechar e reabrir campanhas. Não permitir remover campanhas com cultivos associados.

Gestão de Cultivos
Registar cultivos por campanha: cultura, variedade, hectares, data de plantio. Não permitir remover cultivos com colheitas associadas.

Gestão de Colheitas

Registar colheitas por cultivo: data, quantidade (kg), preço por kg. O sistema calcula o valor total.

Listagens

Listar produtos, fornecedores, clientes, produtos em armazém (stock > 0) e compras efetuadas. Cada listagem permite ordenação (nome, preço, stock, data, valor).

Compras detalhadas

Apresentar cada compra com todos os dados: cliente, endereço, telefone, produtos, quantidades, valor unitário, subtotal, total e data.

Relatórios

- Clientes ordenados por nome (crescente).
- Clientes que compraram (ordenados por volume).
- Clientes mais assíduos (ranking por número de compras).
- (extra): Clientes sem compras.

Persistência em ficheiro

Gravar automaticamente o estado do sistema ao encerrar o *Tomcat* e restaurá-lo ao arranque. *Endpoint* para gravação e eliminação manual.

V. Módulo Cliente

Auto-registo Registo - Com nome, email (único), palavra-passe, telefone e endereço.

Catálogo Visualizar produtos - com nome, descrição, foto, preço, validade, quantidade disponível, fornecedor e categoria. Filtrar por categoria.

Carrinho Adicionar produtos ao carrinho (com quantidade), atualizar quantidades, remover itens, esvaziar carrinho. O carrinho persiste em sessão.

Compra Solicitar - compra se o produto existir e houver stock suficiente. Caso contrário, o sistema avisa da indisponibilidade.

Emissão de factura - Após confirmar, o sistema:

- Desconta automaticamente a quantidade do stock;
- Emite factura com número sequencial (FT--), dados do cliente, itens, subtotal, IVA (16%) e total;
- Permite impressão da factura.

Histórico - Ver todas as compras anteriores e respetivas facturas.

VI. Regras do negócio

a) Não é possível vender mais do que o stock disponível.
b) Uma venda desconta o stock de **todos** os itens, atomicamente (tudo ou nada).
c) Um cliente só vê as **suas** facturas; o *admin* vê todas.
d) Rotas `/admin/*` são exclusivas do administrador (HTTP 403 caso contrário).
e) Rotas `/catalogo`, `/carrinho/*`, `/compra`, `/factura`, `/minhas-compras` exigem sessão.
f) IDs de entidades são sequenciais e únicos, gerados por `AtomicInteger`.

VII. Requisitos não funcionais

- ✓ Interface web em português, responsiva, com CSS próprio.
- ✓ Arquitetura **MVC** clara (*Model, Service, Servlet, JSP*).
- ✓ Operações de escrita concorrente protegidas por `synchronized`.
- ✓ Nenhuma password em texto puro em ambiente de produção (uso de *hash* é desejável).
- ✓ Código organizado em pacotes por responsabilidade (`model`, `repo`, `service`, `servlet`, `filter`, `util`, `listener`).
- ✓ Persistência tolerante a falhas (escrita atómica em ficheiro).
- ✓ Compatibilidade com *Apache Tomcat* 10.1 e Java 17 (de preferência).

VIII. Tecnologías

Camada	Tecnologia
Linguagem	Java 17 (ou superior)
Servidor de aplicações	Apache Tomcat 10.1
Vista	JSP + JSTL + Expression Language + HTML5 + CSS3
Controladores	Jakarta Servlets 6.0 (anotados com @WebServlet)
Filtros	jakarta.servlet.Filter (@WebFilter)
Listeners	jakarta.servlet.ServletContextListener (@WebListener)
Modelos	POJOs Java Serializable
Camada de persistência	ArrayList / Map / Collections.synchronizedList
Camada de persistência	ArrayList / Map / Collections.synchronizedList

Persistência em disco	ObjectOutputStream / ObjectInputStream
Build	Apache Maven (empacotamento WAR)
Proibido: base de dados relacional (MySQL, PostgreSQL) nesta versão. Todos os dados vivem em memória e são opcionalmente serializados para ficheiro.	

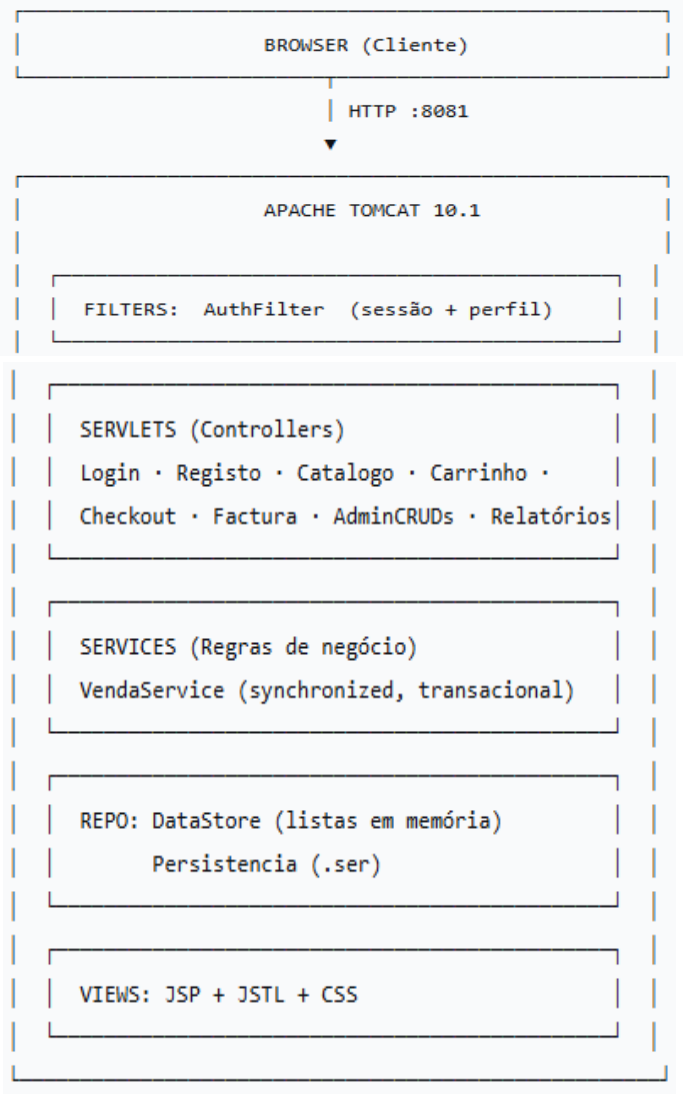
Exemplo de fluxo — Administrador

- *Login: admin@agro.mz / admin234*
- *Dashboard: 3 fornecedores | 3 produtos | 5 clientes | 4 vendas | 1 campanha*
- *Menu Fornecedores → criar "Sementes do Sul, Lda" (tipo SEMENTES, Maputo)*
- *Menu Produtos → criar "Semente Feijão nhemba", categoria SEMENTE, preço 320 MT, quantidade 400 kg, validade +8 meses, fornecedor "Sementes do Sul"*
- *Menu Campanhas → criar "Campanha 2026/2027", 01/10/2026 → 30/06/2027*
- *Menu Cultivos → registar cultivo "Feijão nhemba", variedade "Macia", 8 ha, plantio 15/11/2026*
- *Menu Colheitas → registar colheita 3200 kg a 22 MT/kg (total 70.400 MT)*
- *Menu Listagens → Produtos | Fornecedores | Clientes | Armazém | Compras detalhadas*
- *Menu Relatórios → ver clientes ordenados, clientes que compraram, mais assíduos*
- *Menu Persistência → Gravar agora.*

Exemplo de fluxo — Cliente

- 1. *Registar: joao@teste.mz / 1234 — João Silva, Av. Eduardo Mondlane, Nampula*
- *2. Catálogo: ver produtos com fotos, preços, stock*
- *3. Adicionar 3 kg de Milho ao carrinho*
- *4. Adicionar 1 saco de Adubo ao carrinho*
- *5. Carrinho: ver linhas, alterar quantidades, remover itens*
- *6. Finalizar compra → Confirmar → Sistema emite factura FT-2026-1*
- *7. Factura: dados pessoais + itens + subtotal + IVA 16% + total*
- *8. Imprimir factura*
- *9. Minhas compras: histórico com todas as compras anteriores*

IX. Arquitetura



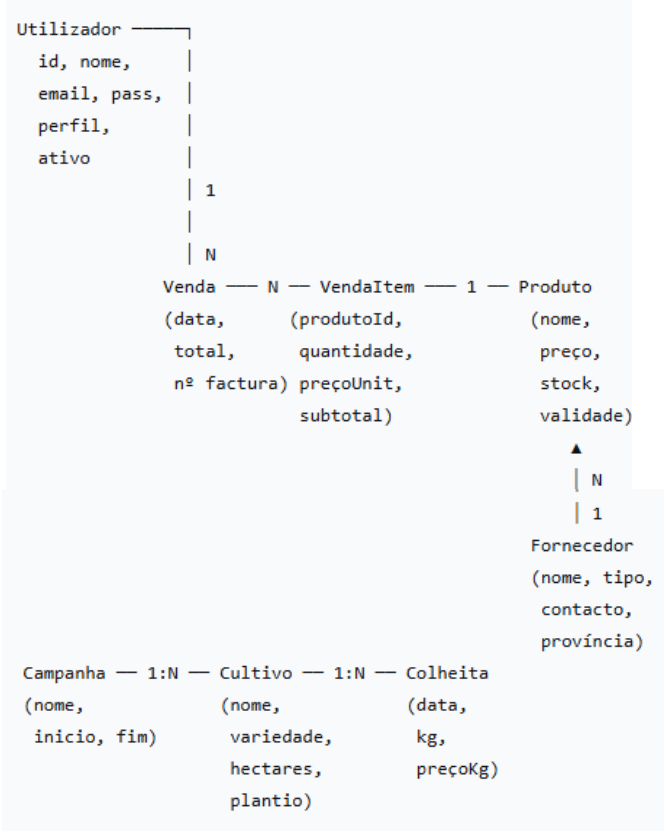
X. Critérios de Avaliação

Critério	Peso
Funcionalidades implementadas corretamente	35%
Arquitetura MVC + organização de código	15%
Interface e usabilidade	10%
Validações e regras de negócio (RN01–RN06)	15%
Persistência (ficheiro) e tolerância a falhas	10%
Deploy correto no Tomcat + operação via systemd	5%
Documentação (README + manual de utilização)	10%

XI. Entregáveis

- Código-fonte completo (repositório Git ou ZIP).
- Ficheiro WAR pronto a publicar.
- README.md com:
- Instruções de instalação e deploy;
- Credenciais de teste;
- Descrição da arquitetura.
- Manual de utilização (PDF) — passo a passo das principais operações para *admin* e cliente.
- Apresentação e defesa com demonstração ao vivo.

XII. Modelo de Domínio



Bom trabalho!